

Credit Card Fraud Detection

Decision Trees vs. Support Vector Machines (SVM)

Duration: 15–20 Minutes | Level: Absolute Beginner | Topic: Machine Learning

What Will We Learn Today?

- What SVM is, in plain English, with a real-life analogy — no jargon
- Every mathematical idea used in SVM, explained as 'what it means in real life' — not how to calculate it by hand
- The real-world problem: catching fraud in 284,807 credit card transactions
- Every single line of Python code, explained in detail, with extra small examples
- How to train and evaluate both a Decision Tree and an SVM, and how to compare them
- At the end: how to DECIDE whether SVM is the right tool for a given dataset

Section 1: The Real-World Problem

Imagine you work at a bank. Every day, lakhs of credit card transactions happen. A tiny number of them are FRAUD — someone stole a card and is misusing it. Your job: build a system that automatically flags a transaction as fraud or not fraud.

The Dataset We Are Using:

284,807 total transactions

Only 492 of them are FRAUD (just 0.172%) — this is called an IMBALANCED dataset

Each transaction has 30 input features (V1 to V28, plus Time and Amount)

V1–V28 are 'disguised' numbers — for privacy, the bank hid the real meaning of each column

The target column 'Class' = 1 (fraud) or 0 (legitimate) — this is what we want to predict

Why does 'imbalanced' matter? If 99.8% of transactions are legitimate, a lazy model could just always guess 'not fraud' and still be right 99.8% of the time! But it would catch ZERO actual fraud cases — completely useless. So we need special techniques to force the model to pay attention to the rare fraud cases. We will see this in the code soon (`sample_weight`).

Section 2: What Is a Support Vector Machine (SVM)?

Forget computers for a second. Imagine you have a big table, and you scatter red marbles and blue marbles all over it randomly — but red marbles tend to stay on one side, and blue marbles tend to stay on the other side.

Real-Life Analogy — Drawing a Border:

Now imagine you have a ruler, and you want to draw ONE straight line on the table that best separates the red marbles from the blue marbles.

You wouldn't draw the line touching the marbles. You would draw it right in the MIDDLE, as far away as possible from the closest red marble AND the closest blue marble.

That is EXACTLY what SVM does — except instead of marbles on a table, it works with data points in many dimensions (30 dimensions in our fraud dataset), and instead of a line, it draws a 'hyperplane' (a fancy word for a flat divider in high dimensions).

So what is SVM used for? SVM is a classification algorithm — it draws a boundary between two (or more) groups so that, given a NEW unseen point, it can tell you which side of the boundary it falls on. It's used heavily in fraud detection, spam email detection, face detection, handwriting recognition, and medical diagnosis — basically anywhere you need to separate things into clear categories.

The Key Idea: The 'Widest Street' Approach

SVM doesn't just draw ANY line that separates the groups — it specifically tries to draw the line that leaves the WIDEST possible gap (street) between the two groups. Why? Because a wider gap means the model is more confident, and it will generalize better to new data it hasn't seen before.

Mathematical Idea #1: The Margin

What is the 'Margin' and why do we care?

REAL-WORLD MEANING:

The margin is simply the WIDTH of that empty street between the two groups of points. It is the distance from the dividing line to the closest point on EITHER side.

WHY DO WE NEED TO CALCULATE IT?

Because SVM's entire goal is to make this margin as WIDE as possible. A wider margin means the model has more 'breathing room' and is less likely to misclassify new, slightly different data in the future. A narrow margin means the model is on edge — even a tiny new data point could end up on the wrong side.

Think of it like a goalkeeper standing exactly in the middle of the goal post — standing in the middle (maximum margin) gives the best chance of saving shots from either side, rather than standing too close to one post.

Mathematical Idea #2: Support Vectors

What are 'Support Vectors' and why do we care?

REAL-WORLD MEANING:

Support vectors are simply the few data points that are CLOSEST to the dividing line — the marbles right at the edge of the street, touching the curb.

WHY DO WE NEED THEM?

Here's the surprising part: SVM does NOT care about most of your data points at all! Only the closest points (the support vectors) actually decide WHERE the dividing line goes. If you removed every other point and kept ONLY the support vectors, the line SVM draws would not change one bit.

This is why SVM is memory-efficient and works well even with a lot of features — it focuses only on the 'borderline cases', not every single data point.

Mathematical Idea #3: Hinge Loss

What is 'Hinge Loss' and why do we care? (we use loss='hinge' in our code)

REAL-WORLD MEANING:

Loss is just a penalty score — how 'wrong' or 'unhappy' the model is with its current dividing line. Hinge loss is one specific way to measure that penalty, designed specifically for SVMs.

HOW DOES IT THINK?

- If a point is correctly classified AND comfortably far from the line → penalty = 0 (no punishment, all good)
- If a point is correctly classified but TOO CLOSE to the line (inside the margin) → small penalty (you're cutting it close!)
- If a point is on the WRONG side of the line entirely → big penalty

WHY DO WE NEED IT?

During training, the computer tries thousands of possible dividing lines and uses hinge loss to score each one. It then picks the line with the LOWEST total penalty — that becomes our final trained model.

Mathematical Idea #4: ROC-AUC Score (used to evaluate our model)

What is 'ROC-AUC' and why do we calculate it?

REAL-WORLD MEANING:

ROC-AUC answers one simple question: 'How good is my model at telling fraud apart from non-fraud, across ALL possible confidence thresholds?'

Imagine the model gives every transaction a 'fraud score' from 0 to 1 (closer to 1 = more suspicious). You then need to pick a cutoff: anything above the cutoff is

flagged as fraud. ROC-AUC checks how well the model separates fraud scores from non-fraud scores — REGARDLESS of which exact cutoff you pick.

HOW TO READ THE NUMBER (it ranges from 0 to 1):

1.0 = Perfect — the model always ranks fraud cases higher than non-fraud cases

0.5 = Useless — same as randomly guessing (flipping a coin)

0.0 = Perfectly backwards (somehow worse than random)

WHY USE THIS INSTEAD OF SIMPLE 'ACCURACY'?

Remember our dataset is 99.8% legitimate transactions. A model that ALWAYS says 'not fraud' would score 99.8% accuracy but be USELESS. ROC-AUC does not get fooled by this imbalance — it specifically checks how well fraud and non-fraud are RANKED and separated, making it the right metric for imbalanced problems like ours.

Section 3: Importing Libraries — Line by Line

```
from __future__ import print_function
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import normalize, StandardScaler
from sklearn.utils.class_weight import compute_sample_weight
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import roc_auc_score
from sklearn.svm import LinearSVC
import warnings
warnings.filterwarnings('ignore')
```

Line 1: from __future__ import print_function

What it does: This is an old-style compatibility line. It makes sure print() works the same way even on very old versions of Python (Python 2). In modern Python (3.x), this line does nothing harmful — it's just a safety habit some instructors still teach. You can usually skip it today.

Lines 2-3: import pandas as pd / import matplotlib.pyplot as plt

pandas (pd): The library for working with data tables (like Excel sheets) — loading, filtering, and exploring data.

matplotlib.pyplot (plt): The library for drawing charts and graphs.

Quick example:

```
import pandas as pd
df = pd.read_csv('data.csv') # load a table from a file
```

```
print(df.shape)          # e.g. (284807, 31) -> rows, columns
```

Line 4: %matplotlib inline

What it does: A special Jupyter Notebook command that displays charts directly below the code cell, instead of in a separate pop-up window. Only works inside Jupyter/Colab notebooks.

Line 5: from sklearn.model_selection import train_test_split

What it does: Imports a function that splits our dataset into a TRAINING part (model learns from this) and a TESTING part (we check the model's performance on this, since it has never seen this data before).

Line 6: from sklearn.preprocessing import normalize, StandardScaler

StandardScaler: A tool that rescales every number column so it has an average of 0 and a 'spread' of 1. Imagine some columns are in rupees (Amount: 0 to 50,000) and others are tiny decimals (-2 to 2). Without scaling, the model would think the rupee column is 'more important' just because the numbers are bigger — which is wrong! Scaling puts everyone on a level playing field.

normalize: A function that adjusts each ROW of data so its values follow a consistent total 'size' (we use the L1 norm — explained below). This helps the SVM's maths converge (settle on an answer) faster and more reliably.

What is 'L1 Norm' (used in normalize(X, norm='l1')) and why do we need it?

REAL-WORLD MEANING:

Imagine each transaction's 29 feature values as ingredients in a recipe, in different proportions. The L1 norm is just the SUM of the absolute (always-positive) values of all the ingredients in that one row.

'Normalizing with L1' means: divide every value in the row by that row's total sum, so that all the values in the row now add up to exactly 1 (like converting amounts into percentages of a whole).

WHY DO WE NEED IT?

Some transactions naturally have bigger or smaller raw numbers than others. By converting every row to the same 'scale' (proportions adding to 1), we make sure the model focuses on the RELATIVE pattern between features within a transaction, rather than getting confused by one transaction simply having bigger raw numbers overall. This also helps SVM's internal calculations finish faster and avoid getting stuck.

Line 7: `from sklearn.utils.class_weight import compute_sample_weight`

What it does: Imports a tool that calculates how much 'extra attention' the model should give to each row, based on how rare its class is. Since fraud cases are only 0.172% of our data, this tool will tell the model: 'pay much more attention to each fraud example, because there are so few of them!'

Line 8: `from sklearn.tree import DecisionTreeClassifier`

What it does: Imports the Decision Tree algorithm — the same one from your previous lesson! It asks a series of YES/NO questions about the data to reach a final answer (fraud or not fraud).

Line 9: `from sklearn.metrics import roc_auc_score`

What it does: Imports the function that calculates the ROC-AUC score we discussed earlier — our main scorecard for comparing the Decision Tree and SVM.

Line 10: `from sklearn.svm import LinearSVC`



Syntax:

LinearSVC = Linear Support Vector Classifier

This is the specific SVM tool we use — it draws a STRAIGHT (linear) dividing boundary.

What it does: Imports the SVM algorithm itself, specifically the 'linear' version, meaning it tries to separate fraud from non-fraud using a straight-line-style boundary (in 29-dimensional space).

Lines 11-12: `import warnings / warnings.filterwarnings('ignore')`

What it does: Hides non-critical warning messages so the notebook's output stays clean and easy to read for beginners.

Section 4: Loading the Dataset

```
url = "https://cf-courses-data.s3.us.cloud-object-  
storage.appdomain.cloud/.../creditcard.csv"  
raw_data = pd.read_csv(url)  
raw_data
```

What happens: `pd.read_csv(url)` downloads the CSV file directly from the internet link and loads it into a pandas table called `raw_data`. Typing '`raw_data`' alone on the last line displays a preview of the table (first and last few rows) — a habit Jupyter notebooks support automatically for the last line of a cell.

Visualizing the Class Imbalance

```
labels = raw_data.Class.unique()
sizes = raw_data.Class.value_counts().values

fig, ax = plt.subplots()
ax.pie(sizes, labels=labels, autopct='%1.3f%%')
ax.set_title('Target Variable Value Counts')
plt.show()
```

raw_data.Class.unique(): Looks at the 'Class' column and returns the distinct values found — in our case, just [0, 1].

raw_data.Class.value_counts(): Counts how many rows belong to each class. .values converts this into a plain list of numbers, e.g. [284315, 492].

Syntax:

fig, ax = plt.subplots() -> Creates an empty figure (fig) and a drawing area (ax)
ax.pie(sizes, labels=..., autopct='%1.3f%%') -> Draws a PIE CHART
 sizes -> the slice sizes (counts of each class)
 labels -> text label for each slice
 autopct -> format string to show percentage on each slice (3 decimal places here)
ax.set_title('...') -> Sets the chart title
plt.show() -> Displays the chart

Simple pie chart example:

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
ax.pie([70, 30], labels=['Pass', 'Fail'], autopct='%1.1f%%')
ax.set_title('Exam Results')
plt.show()
# This draws a pie chart with 70% Pass and 30% Fail slices
```

Section 5: Checking Which Features Matter Most

```
correlation_values = raw_data.corr()['Class'].drop('Class')
correlation_values.plot(kind='barh', figsize=(10, 6))
```

raw_data.corr(): Calculates the correlation between EVERY pair of columns in the table — a number from -1 to +1 showing how strongly two columns move together (explained back in your Decision Tree lesson).

['Class']: From that big correlation table, we select only the column showing each feature's correlation WITH the 'Class' (fraud) column — this tells us which features are most linked to fraud.

.drop('Class'): Removes the 'Class vs Class' row (which is always exactly 1.0, since anything is perfectly correlated with itself) — that row is not useful information.

.plot(kind='barh', figsize=(10,6)): Draws a HORIZONTAL bar chart (barh = bar-horizontal) of these correlation values, sized 10 inches wide by 6 inches tall, so we can visually see which features have the strongest positive or negative link to fraud.

Section 6: Preparing the Data for Training

```
raw_data.iloc[:, 1:30] = StandardScaler().fit_transform(raw_data.iloc[:,
1:30])
data_matrix = raw_data.values

X = data_matrix[:, 1:30]
y = data_matrix[:, 30]

X = normalize(X, norm='l1')
```

raw_data.iloc[:, 1:30]



Syntax:

`df.iloc[ row_selection, column_selection ]`

'.' alone means 'ALL rows'

'1:30' means 'columns from index 1 up to (but not including) index 30'

.iloc stands for 'integer location' — selecting by POSITION NUMBER, not by name.

Why columns 1 to 30 (not 0)? Column index 0 is 'Time' — the notebook explicitly says we are NOT using Time as an input feature. So we skip it and grab columns 1 through 29 (V1 to V28, plus Amount), leaving column 30 (Class) for later.

Example of iloc slicing:

```
import pandas as pd
df = pd.DataFrame({'A':[1,2], 'B':[3,4], 'C':[5,6], 'D':[7,8]})
print(df.iloc[:, 1:3])
# Selects columns B and C (index 1 and 2) for ALL rows
```

StandardScaler().fit_transform(...)



Syntax:

`StandardScaler()` -> creates a new scaler tool

`.fit_transform(data)` -> learns the mean & spread of the data, then rescales it

What happens: Every value in columns 1 to 29 gets rescaled so that, overall, each column ends up with an average of 0 and a standard 'spread' of 1. This is the same StandardScaler concept explained in Section 3 — now actually being applied to our real data.

Mini example of standard scaling:

```
from sklearn.preprocessing import StandardScaler
import numpy as np
data = np.array([[10],[20],[30]])
scaled = StandardScaler().fit_transform(data)
print(scaled)
# Output roughly: [[-1.22], [0.0], [1.22]]
# The middle value (20, the average) becomes 0
```

data_matrix = raw_data.values

What it does: .values converts our pandas table into a plain NumPy array (just rows and columns of pure numbers, no labels). Many scikit-learn functions prefer working with this raw number format.

X = data_matrix[:, 1:30] and y = data_matrix[:, 30]

X (features): Grabs columns 1 to 29 — these are our INPUT variables (the 29 transaction details the model will learn from). Note: this is NumPy slicing now, similar to the .iloc slicing seen earlier, just on a plain array instead of a DataFrame.

y (target): Grabs column index 30 — the very last column, 'Class' — this is what we are trying to PREDICT (fraud = 1, not fraud = 0).

X = normalize(X, norm='l1')

What it does: Applies the L1 normalization we explained mathematically in Section 3, to our actual feature matrix X. After this line, every row's values are rescaled to be proportions that sum to 1 (in absolute terms).

Section 7: Splitting Into Training and Testing Sets

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)
```



Syntax:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

X, y -> our feature matrix and target column
test_size=0.3 -> 30% of data goes to testing, 70% to training

```
random_state=42 -> a fixed 'seed' so the split is identical every time the code runs
```

What happens: Out of 284,807 transactions, about 199,365 go into training (the model learns the patterns from these) and about 85,442 go into testing (used only to check performance afterward, never seen during training).

Section 8: Handling the Imbalanced Data (Sample Weights)

```
w_train = compute_sample_weight('balanced', y_train)
```

What it does: Remember, only 0.172% of our data is fraud. Without help, the model could 'lazily' ignore fraud cases and still seem accurate. `compute_sample_weight('balanced', ...)` calculates a special weight number for every row: fraud rows get a MUCH HIGHER weight, and non-fraud rows get a LOWER weight, so that during training, the model is forced to pay serious attention to each rare fraud example, as if it occurred more often.

Simple intuition:

Imagine a classroom of 100 students: 98 are quiet, only 2 ever raise concerns. If a teacher ignores the 2 because they're rare, problems get missed. 'Balanced weighting' is like the teacher deciding: 'I will treat each of those 2 students' concerns as being as important as 49 of the quiet students combined' — forcing fair attention despite the small numbers.

Section 9: Training the Decision Tree Model

```
dt = DecisionTreeClassifier(max_depth=4, random_state=35)
dt.fit(X_train, y_train, sample_weight=w_train)
```

DecisionTreeClassifier(max_depth=4, random_state=35): Creates a new Decision Tree model, limited to 4 levels of YES/NO questions (`max_depth=4`), with a fixed `random_state` so results are reproducible. (Refer to your earlier Decision Tree lesson for full detail on how this algorithm thinks.)

dt.fit(X_train, y_train, sample_weight=w_train): Trains the model on our training data. The new part here is `sample_weight=w_train` — this tells the Decision Tree to use the special 'pay more attention to fraud' weights we calculated in Section 8 while learning its rules.

Section 10: Training the SVM Model

```
svm = LinearSVC(class_weight='balanced', random_state=31, loss='hinge',
fit_intercept=False)
```

```
svm.fit(X_train, y_train)
```

Understanding Each Parameter of LinearSVC()



Syntax:

```
LinearSVC( class_weight='balanced', random_state=31, loss='hinge', fit_intercept=False )
```

class_weight='balanced': This is SVM's own built-in way of handling class imbalance — same idea as `compute_sample_weight` from Section 8, but SVM lets us do it directly as one simple setting, with no separate weight calculation needed.

random_state=31: Fixes randomness so we get the same result every time we re-run the code.

loss='hinge': Tells SVM to use the Hinge Loss penalty system we explained mathematically in Section 2 — penalizing points that are misclassified or too close to the dividing boundary.

fit_intercept=False: The 'intercept' is a small adjustment value that shifts the dividing line/boundary up or down. Setting this to False tells SVM: 'assume the boundary should naturally pass through the origin point (zero), don't bother shifting it.' This is a reasonable choice here because our data was already standardized and normalized in Section 6, which centers everything around zero anyway.

svm.fit(X_train, y_train): Trains the SVM model — it searches for the dividing boundary that creates the widest possible margin between fraud and non-fraud transactions, using hinge loss to measure how good each candidate boundary is.

Section 11: Evaluating the Decision Tree

```
y_pred_dt = dt.predict_proba(X_test)[: ,1]

roc_auc_dt = roc_auc_score(y_test, y_pred_dt)
print('Decision Tree ROC-AUC score : {0:.3f}'.format(roc_auc_dt))
```



Syntax:

`model.predict_proba(X_test)` -> Returns PROBABILITIES instead of plain yes/no answers
Returns 2 columns: [probability of class 0, probability of class 1]
`[:, 1]` -> Selects ONLY the second column (probability of FRAUD)

Why probabilities instead of plain 0/1 predictions? ROC-AUC needs to know HOW CONFIDENT the model is, not just its final yes/no guess, so it can test many different confidence cutoffs (as explained in Section 2). `predict_proba` gives us that confidence score, between 0 and 1.

roc_auc_score(y_test, y_pred_dt): Compares the actual true answers (`y_test`) against the model's predicted fraud probabilities (`y_pred_dt`) and calculates the ROC-AUC score discussed in Section 2.



Syntax:

`{0:.3f}.format(value)` -> Python string formatting
`{0}` refers to the first value passed into `.format()`
`:.3f` means 'show as a decimal number with exactly 3 digits after the point'

String formatting example:

```
score = 0.94567123
print('Score: {0:.3f}'.format(score))
# Output: Score: 0.946
```

Section 12: Evaluating the SVM

```
y_pred_svm = svm.decision_function(X_test)

roc_auc_svm = roc_auc_score(y_test, y_pred_svm)
print('SVM ROC-AUC score: {0:.3f}'.format(roc_auc_svm))
```

svm.decision_function(X_test): Unlike Decision Trees, standard LinearSVC does not naturally produce a 0-to-1 probability. Instead, `decision_function` gives a raw 'distance score' — how far (and on which side) each test point lies from the dividing boundary. Points far on the fraud side get a high positive number; points far on the legitimate side get a very negative number.

Why does ROC-AUC still work with this? ROC-AUC only cares about the RANKING/ORDER of scores, not their exact scale (0-to-1 or otherwise). As long as fraud cases tend to get higher scores than non-fraud cases, ROC-AUC can measure that separation correctly.

Section 13: When Should You Actually Use SVM?

This is the big-picture question every data scientist asks before picking a model. Here's how you decide, based directly on what we observed in this very lab:

The Practice Exercise Result (a real built-in experiment!):

What the lab's practice exercises show us:

Using all 29 features:

SVM and Decision Tree both perform reasonably, with SVM doing well with the full richness of feature information available.

Using only the TOP 6 most correlated features (V3, V10, V12, V14, V16, V17):

- Decision Tree ROC-AUC score actually IMPROVES
- SVM ROC-AUC score actually DROPS

This single experiment teaches a major lesson about when each model shines.

How to Decide: SVM vs. Decision Tree

✓ SVM tends to be a GOOD CHOICE when:

- You have MANY features/columns (high-dimensional data) — SVM actually benefits from extra dimensions, since more dimensions make it easier to find a clean separating boundary.
- The classes are roughly separable by a smooth boundary (a line, or curve with a 'kernel' trick) rather than by complicated nested if-else rules.
- You care more about a clean mathematical margin/boundary than about an easily explainable set of rules.
- Your data has been properly scaled/normalized (SVM is sensitive to differences in scale between features — this is WHY we used StandardScaler earlier!).

✓ Decision Tree tends to be a GOOD CHOICE when:

- You have FEWER, more meaningful features, especially after feature selection (like the top-6 correlated features experiment above).
- You want a model that's easy to EXPLAIN to non-technical people (if-this-then-that rules, as you learned in your previous Decision Tree lesson).
- The relationship between features and the answer follows clear threshold-based rules (e.g. 'if Na_to_K > 14.6 then...') rather than smooth geometric separation.
- You don't want to worry much about scaling your features first.

The Takeaway Rule of Thumb

🎯 Quick Decision Rule:

MORE features, less obviously meaningful individually → Lean toward SVM
FEWER, hand-picked, highly meaningful features → Lean toward Decision Tree

In real projects, the best approach (just like in this lab) is to TRY BOTH, measure both with the same metric (ROC-AUC here, since our data was imbalanced), and let the actual numbers decide — rather than assuming one algorithm is always best.

Summary — What Did We Learn Today?

- SVM finds the dividing boundary with the WIDEST possible margin between two classes — like drawing a road right in the middle of two groups of marbles.
- Support vectors are the few 'edge' points that actually decide where that boundary goes; hinge loss is how SVM scores how good a candidate boundary is.

- ROC-AUC measures how well a model separates fraud from non-fraud across all confidence thresholds — perfect for imbalanced datasets like ours.
- StandardScaler and L1 normalization both prepare our features so no single column unfairly dominates the model, and so SVM's maths converges faster.
- Imbalanced data (only 0.172% fraud) needs special handling — `sample_weight` for Decision Trees, `class_weight='balanced'` for SVM.
- `predict_proba()` gives confidence scores for Decision Trees; `decision_function()` gives distance-from-boundary scores for SVM — both work fine with ROC-AUC.
- SVM tends to shine with MANY features; Decision Trees tend to shine with FEWER, well-chosen features — always test both and compare with the same metric.

Great work! You now understand SVM and Decision Trees well enough to choose the right tool for your own dataset.